

软件测试

常用测试分类



测试分类回答三个问题：

- 1. 能看到多少内部信息？**
- 2. 在软件生命周期哪个层级测试？**
- 3. 想验证哪一种质量？**

常用测试分类概述

①

源代码信息依赖程度

白盒测试
灰盒测试
黑盒测试

②

软件开发流程

单元测试
集成测试
系统测试
验收测试

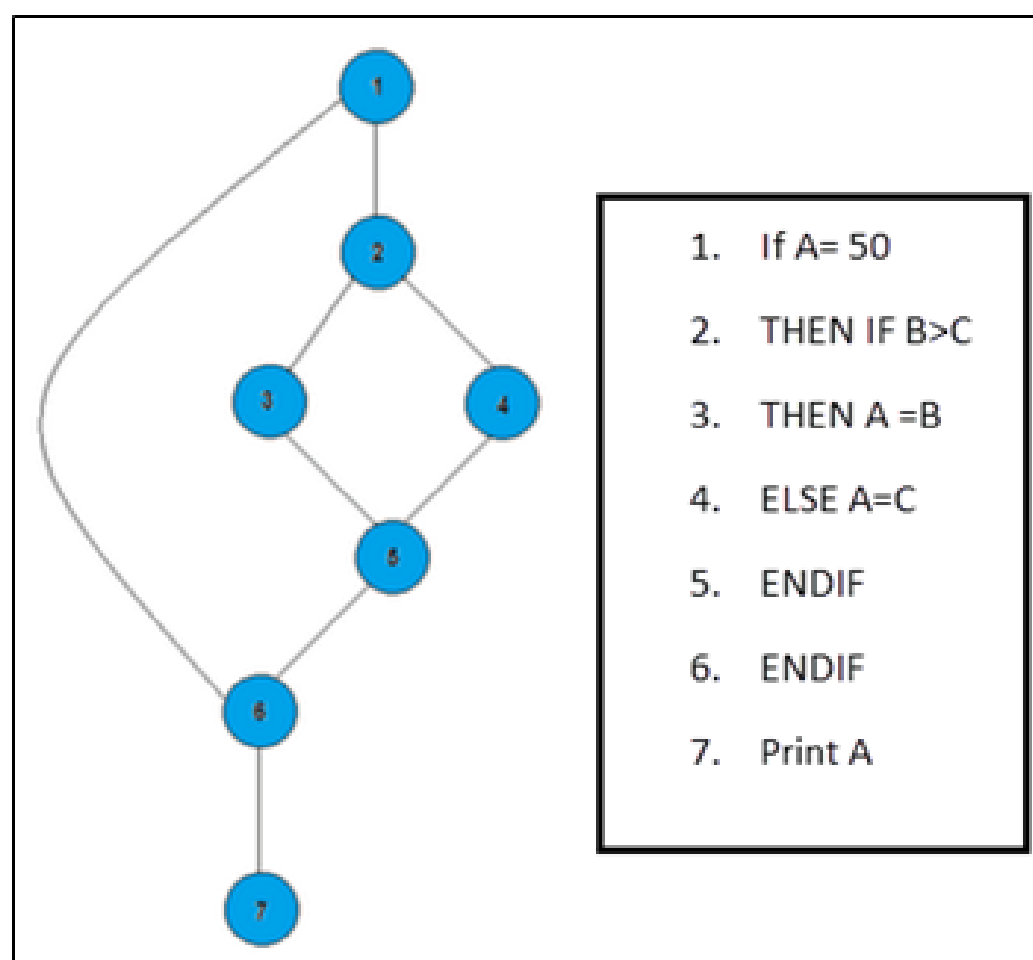
③

软件质量属性

功能测试
性能测试
安全测试
兼容性测试
易用性测试

白盒测试

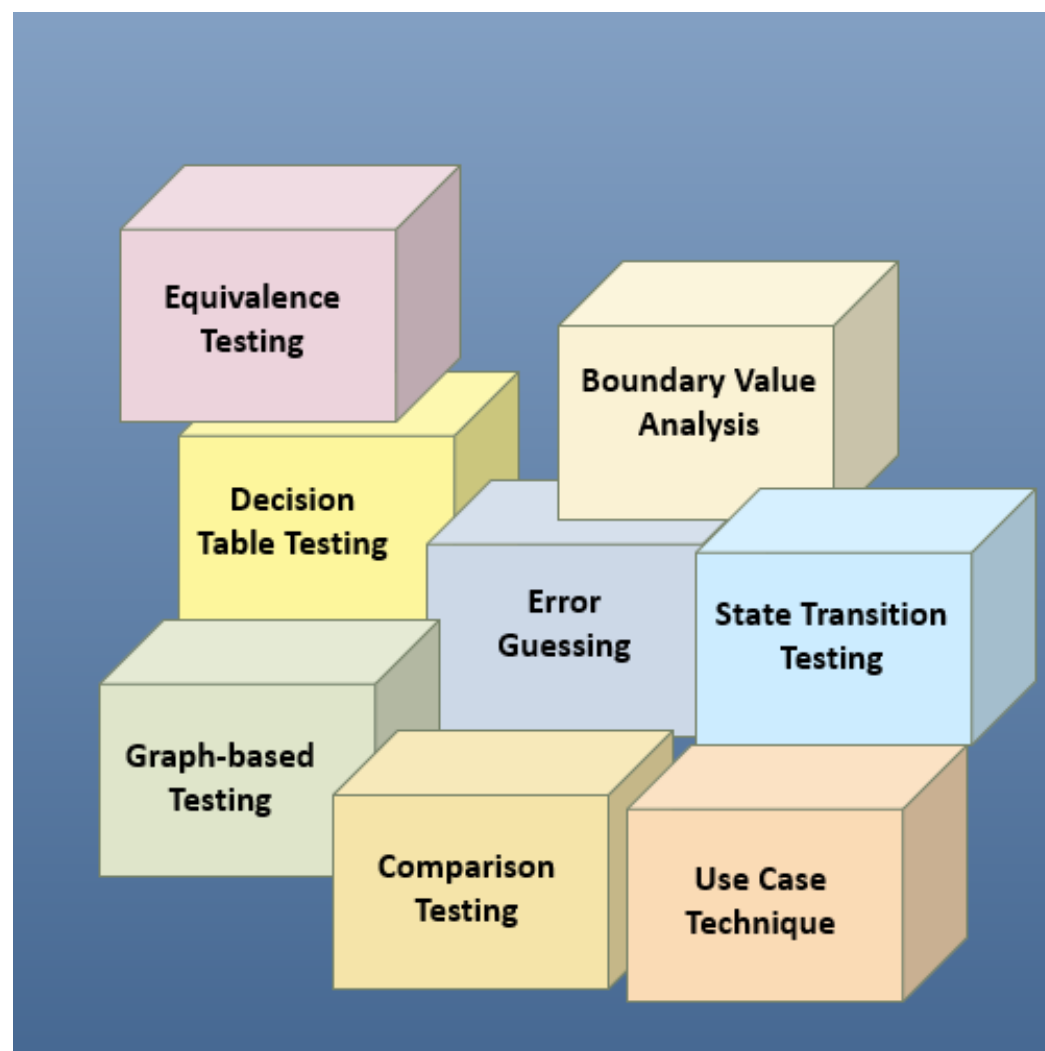
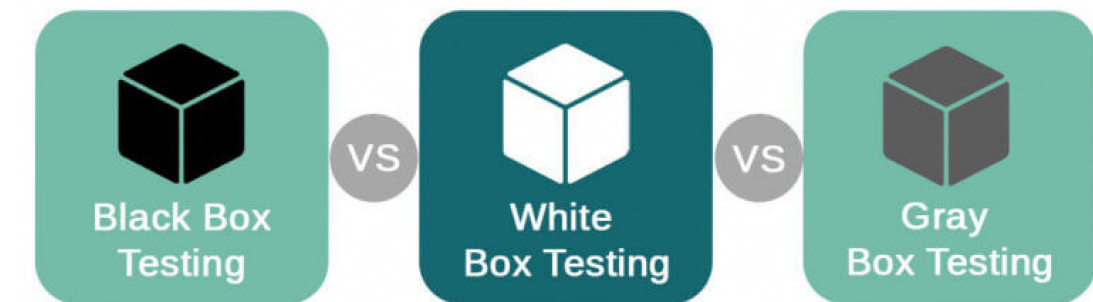
(White-box Testing)



- 白盒测试依赖源代码信息，主要目的是测试应用程序内部的结构和运行情况。
- 白盒测试从程序内部角度出发进行测试用例设计，通过程序路径覆盖特别目标发现程序中潜在缺陷。
- 测试人员需要理解待测试程序的内部结构，这种程序理解的先决性往往使得白盒测试成本高昂。

黑盒测试

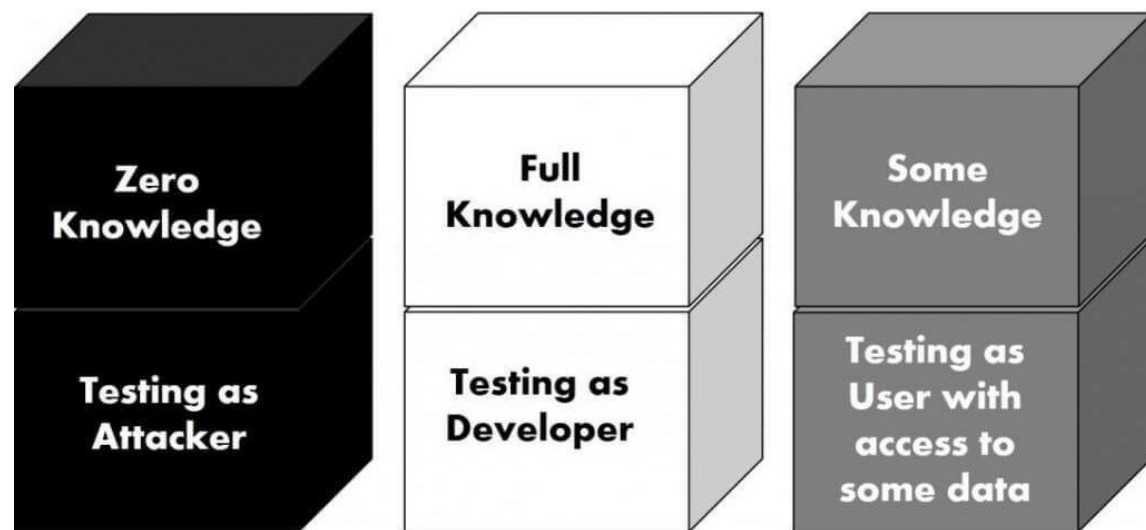
(Black-box Testing)



- 黑盒测试更强调从用户的角度出发，针对软件的界面、功能及外部结构进行测试，通常不考虑程序内部逻辑结构。
- 黑盒测试通常由测试人员根据软件规范、软件规格说明或设计文档设计而成。
- 黑盒测试可用于多功能测试、性能测试、兼容性测试等，具有应用范围广泛、测试效率卓越、测试覆盖全面的优点。

灰盒测试

(Gray-box Testing)

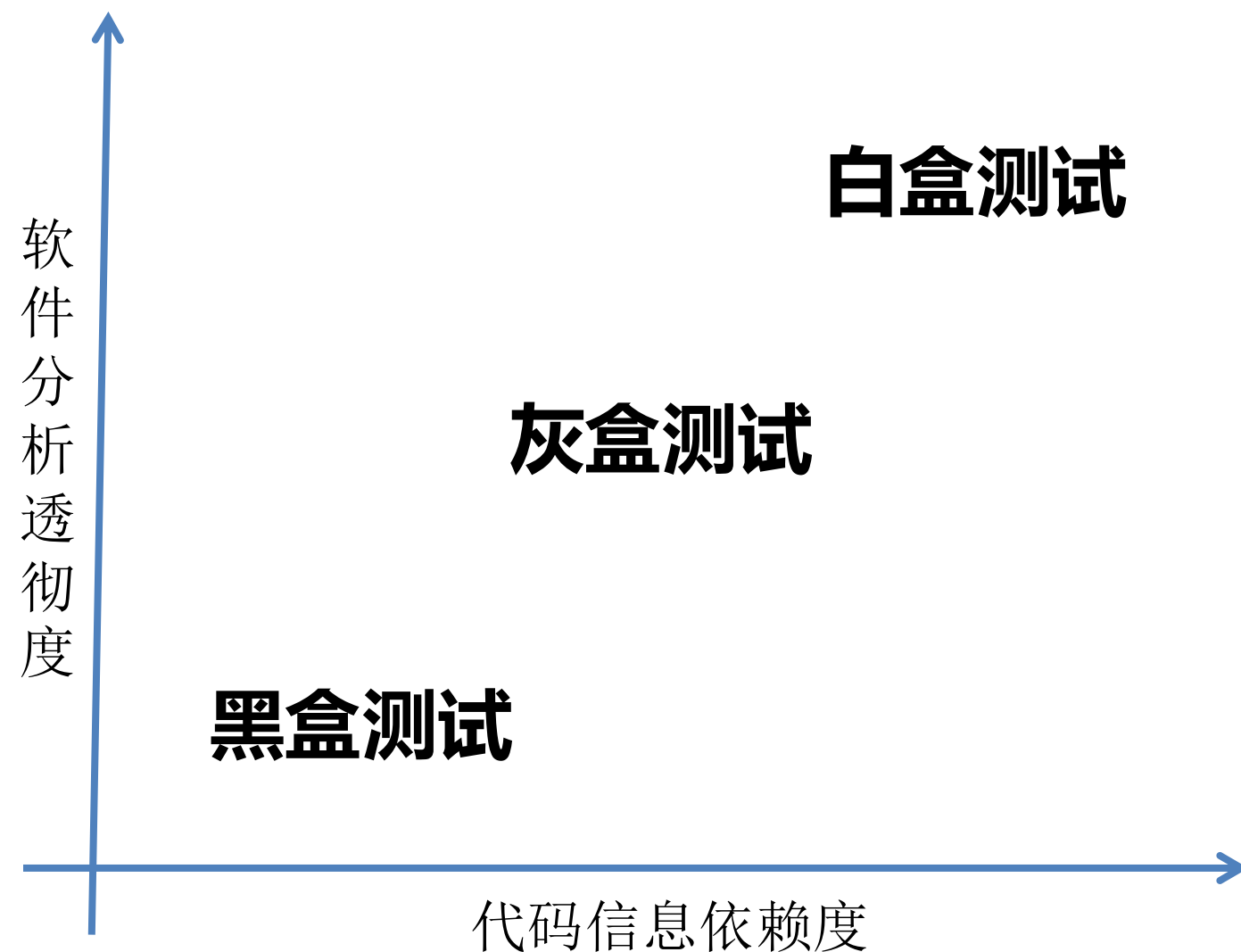


特别注意：灰盒测试不等于白盒测试加黑盒测试！



- 灰盒测试是一种介于白盒和黑盒之间的方法。相对于黑盒，灰盒测试更加注重程序的内部逻辑，但源代码信息不像白盒那样详尽。
- 灰盒测试通过一些表征性的现象或事件来反映内部运行状态。灰盒测试使用特定方法和工具来提取应用程序的内部知识和交互信息，进而使用内部间接信息来设计测试，以提高测试效率。

白盒测试/灰盒测试/黑盒测试

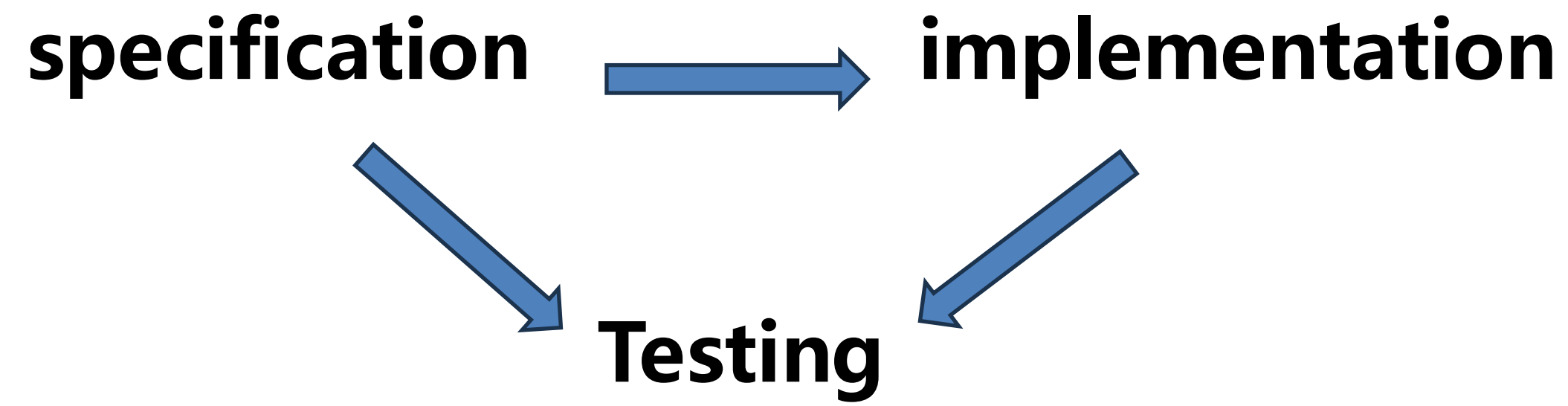


- 从黑盒、到灰盒、再到白盒，对软件源代码的信息依赖程度逐步提高，也常常越来越受限。
- 另外一个维度看，白盒能实现更加透彻的软件分析，其次是灰盒，黑盒能分析的内容较少。
- 软件开发早期，更多依赖白盒测试，软件开发后期，更多依赖黑盒测试。
- 白盒、灰盒、黑盒测试常常相互关联分析，以完成高效的缺陷定位与修复。

讨论：三种测试方法如何关联？

同一个 LLM 先写程序，再读自己写的程序生成测试，有什么问题？

- 它可能把同一个需求理解错误复制两遍；**
- 测试会贴着实现走；**
- 测试可能验证“代码已经做了什么”，而不是“需求应该做什么”。**



按软件开发流程分类

软件开发流程

单元测试
集成测试
系统测试
验收测试

思考：四种测试方法如何关联？

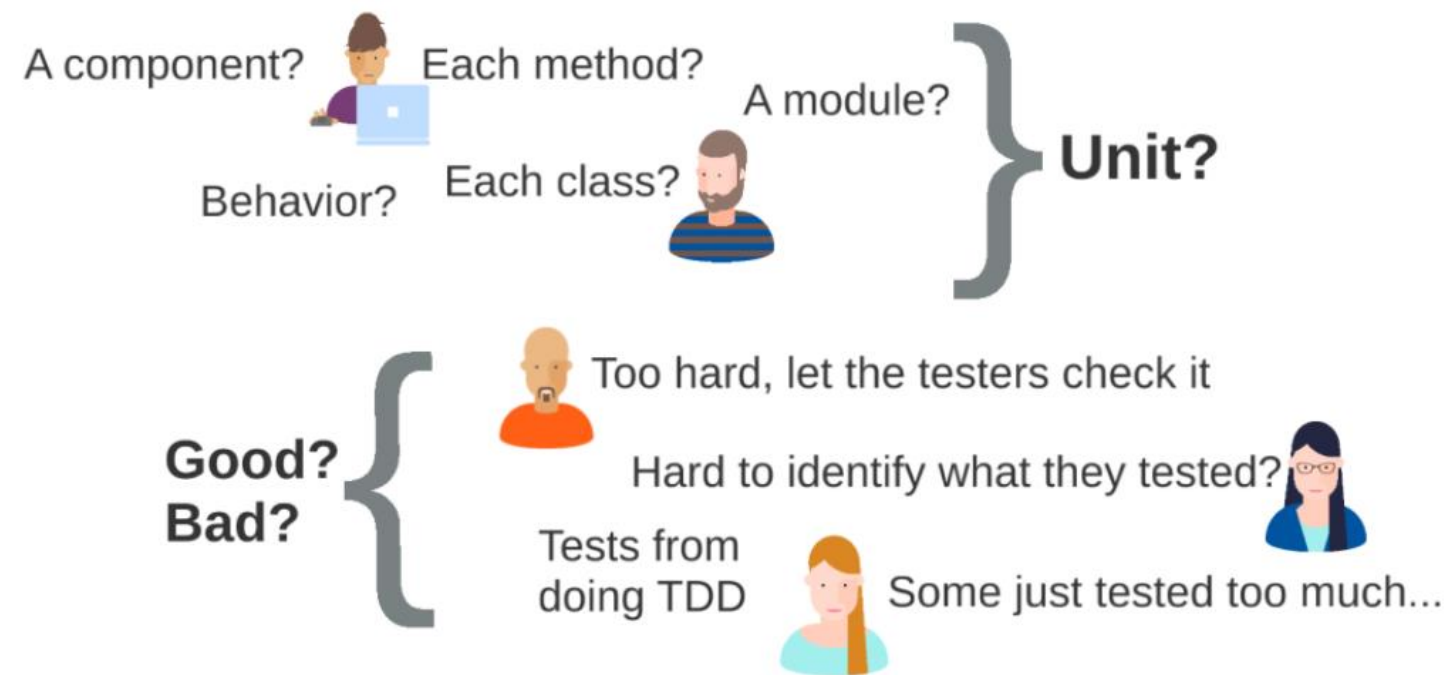


(V-Model)

单元测试

(Unit Testing)

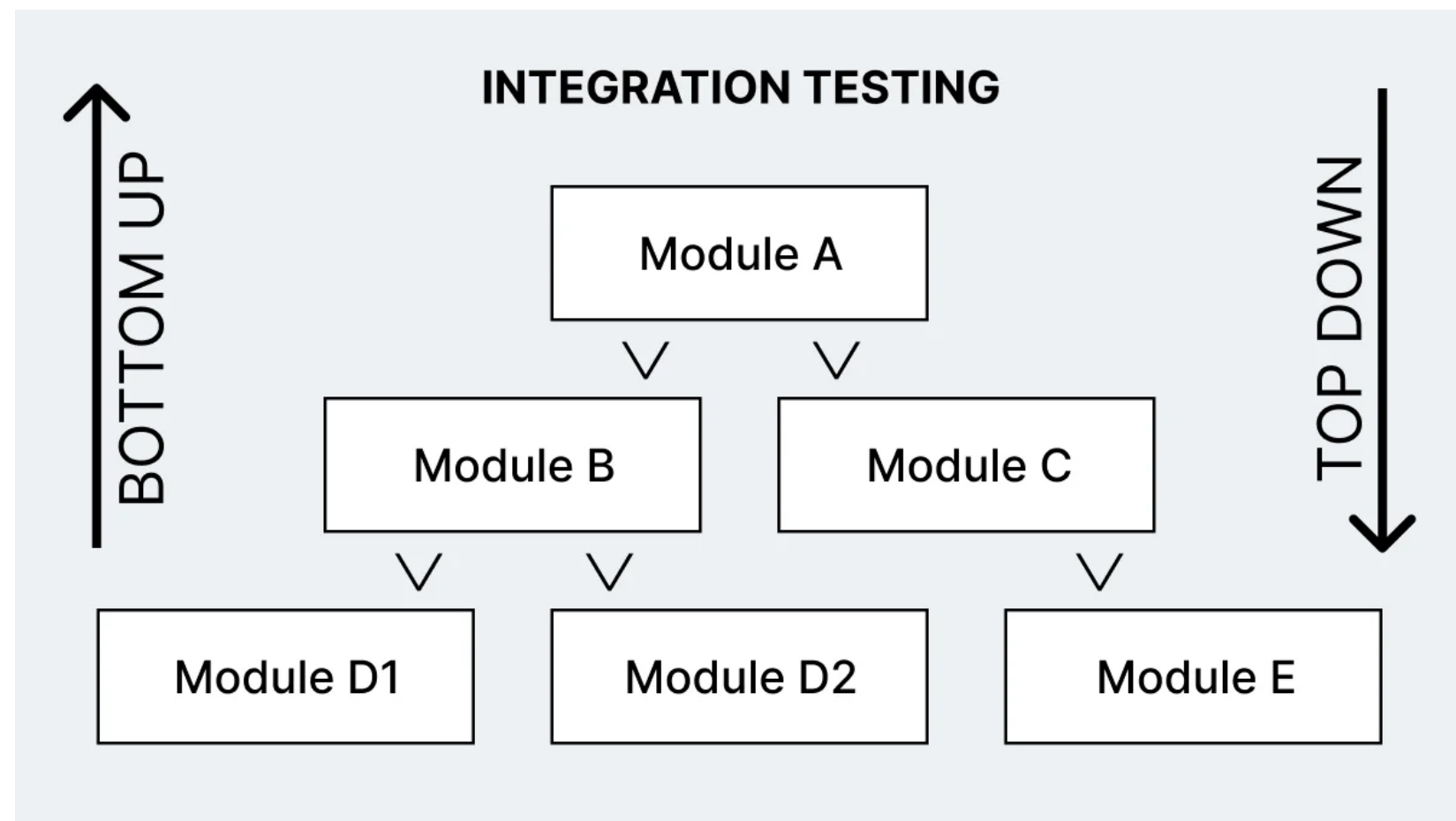
*What do the
Unit tests test?*



- 单元测试是针对软件中最小可测试单元进行验证的技术。单元是指软件中最小可测试部分。一个单元可以是独立的函数和过程。
- 测试人员通常会使用测试框架、驱动程序、模拟对象等方式辅助完成单元测试。单元测试的基本原则是保证测试用例之间相互独立。
- 单元测试一般由软件的开发人员来实施，目的是检验所开发的代码功能是否符合开发者自己规定的设计要求。

集成测试

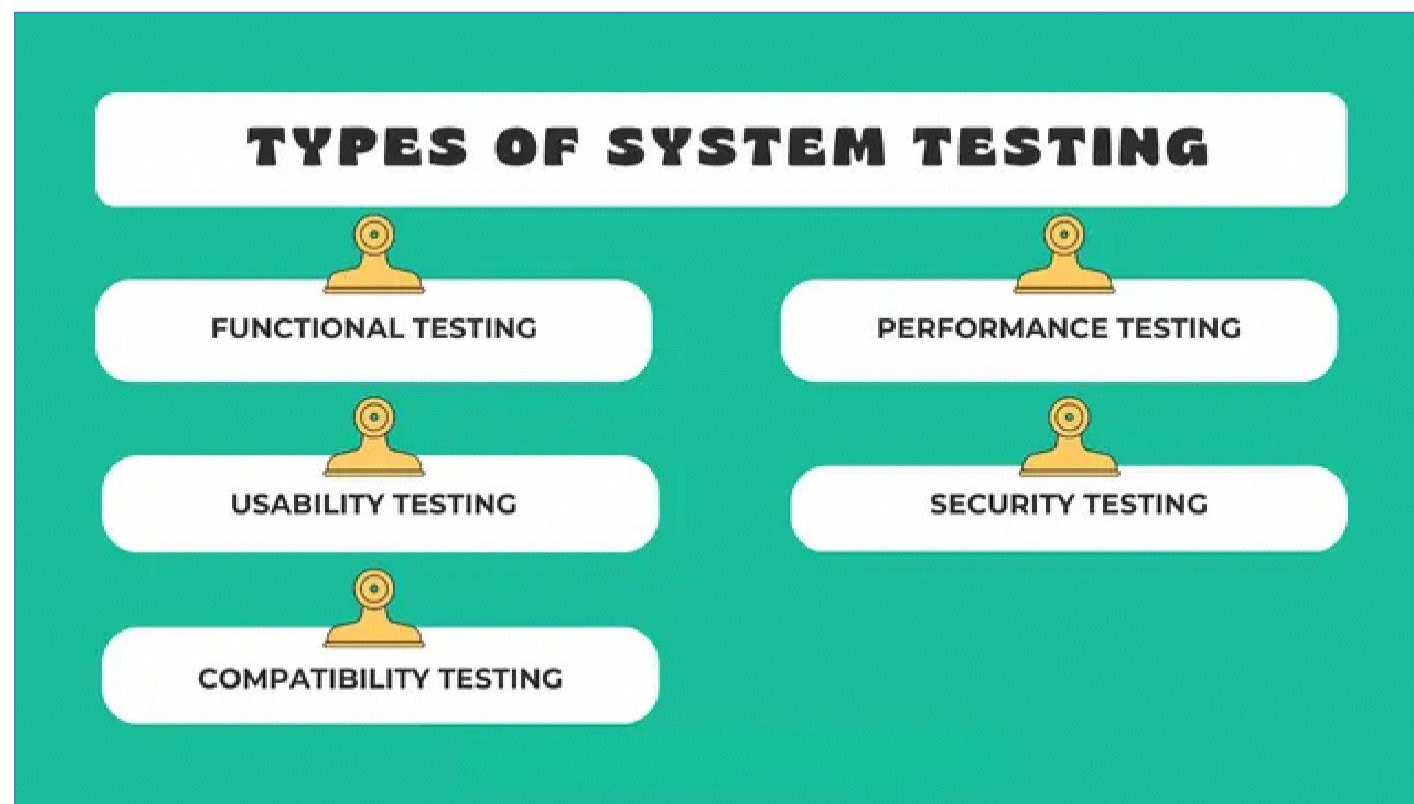
(Integration Testing)



- 集成测试将在所有的软件单元按照概要设计规格说明要求组装成模块、子系统或系统，确保各个单元部分之间的协作良好。
- 在集成测试之前，应该先完成单元测试。集成测试将已经测试的多个单元组装成一个局部整体进行测试。这种测试用来集成测试的目的是确认在不同单元模块之间的交互中没有出现问题。

系统测试

(System Testing)



- 系统测试是针对整个系统的测试，它将与系统相关的硬件与软件看作一个整体，检验系统是否符合预期。
- 系统测试的关注重点包括待测系统本身的使用，待测系统与相关系统间的连通，以及待测系统在真实使用环境下的表现等。
- 系统测试则侧重整个系统的功能验证，更加靠近业务端。除了功能测试，系统测试还包括性能测试、安全测试、兼容性测试、稳定性测试等。

验收测试

(Acceptance Testing)



- 验收测试也称交付测试，是针对用户需求和业务流程的测试，其目的是验证系统是否满足验收标准，并由用户或其他授权机构决定是否接受该系统。
- 验收测试可以分成内部验收测试和外部验收测试。内部和外部验收测试的区别主要在于测试的参与者不同。测试执行者可以是软件开发团队中软件开发和软件测试活动的非直接参与者，也可以是软件的最终用户和潜在用户扮演者，在某些情况下可采取众包方式来实现。

软件质量属性

软件质量属性

功能测试

性能测试

安全测试

兼容性测试

易用性测试

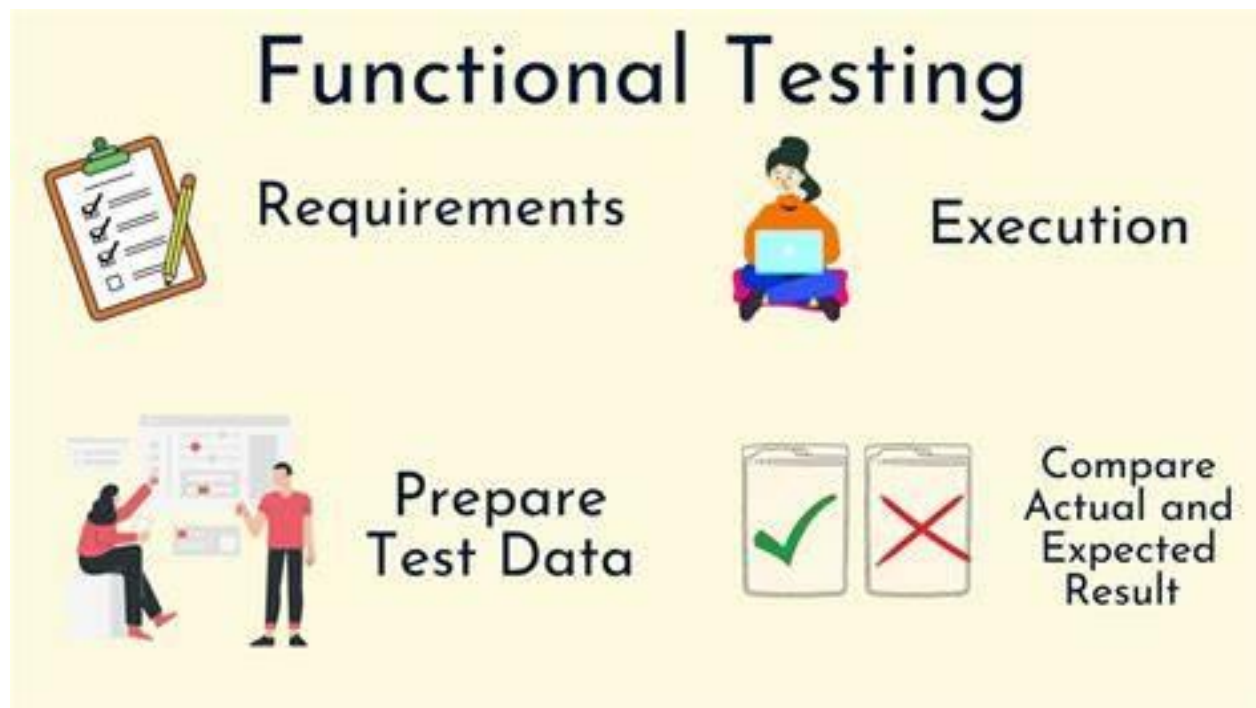
产品质量属性是指用于描述和评估产品质量的各种特性，它是衡量产品满足规定要求和用户需求程度的重要指标。

软件产品质量属性除了具有一般产品的共性外，还具有以下特征：

- 复杂性（Complexity）：软件要解决的问题通常是一种人为抽象化的智能活动，从而导致软件是复杂的。
- 顺从性（Conformity）：软件各部分必须协同一致，由于时间和环境的演变，要维持这样的一致性通常十分困难。
- 易变性（Changeability）：软件容易受外部环境多种因素影响，而且这些因素常常快速变化。
- 不可见性（Invisibility）：尚未完成的软件是看不见的，也常无法充分呈现其结构，使得人们在沟通上面临极大的困难。

功能测试

(Functional Testing)



黑盒测试，基于需求，可重复性，独立性（开发者偏见）；界面测试，功能点测试，业务流程测试；人工测试，自动化测试

- **验证功能正确：** 确保软件系统的各项功能按照需求规格说明书的要求正常工作，能够准确地实现预期的功能，满足用户的业务需求。
- **发现功能缺陷：** 通过各种测试手段，尽可能地找出软件系统中存在的功能缺陷或错误，以便开发人员及时修复，提高软件的质量和稳定性。
- **增强用户信心：** 经过全面的功能测试，软件在上线前能够尽可能地减少功能方面的问题，从而增强用户对软件的信心，提高用户满意度。

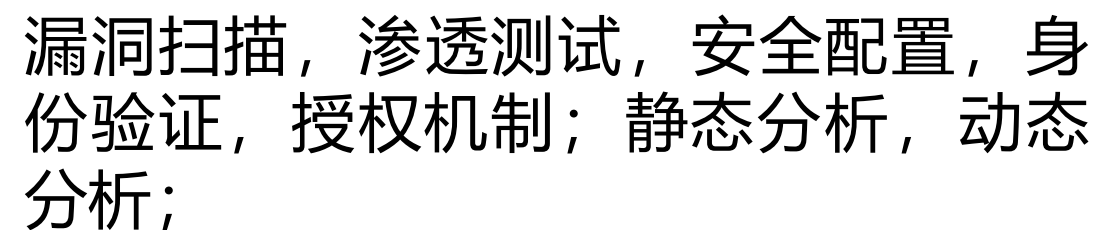
功能测试

(Performance Testing)



响应时间，吞吐量，并发测试，资源利用率；性能测试工具，资源监控工具；环境模拟，数据驱动，多维分析

- **评估性能指标：**通过性能测试，获取系统在响应时间、吞吐量、资源利用率等方面的具体指标数据，以确定系统当前的性能水平是否满足用户需求和业务要求。
- **发现性能瓶颈：**找出系统在硬件、软件、网络等方面存在的性能瓶颈，以便开发和运维团队有针对性地进行优化。
- **优化系统性能：**在性能测试过程中，通过对不同场景下性能数据的分析，为系统优化提供依据。
- **支持容量规划：**根据性能测试的结果，预测系统在未来不同业务量增长情况下的性能表现，为系统的容量规划提供参考。



- **发现安全漏洞：**检测软件系统中可能存在的安全漏洞，防止攻击者利用这些漏洞获取敏感信息、篡改数据或破坏系统。
- **评估安全风险：**评估软件系统面临的安全风险，确定安全漏洞可能造成的影响和危害程度，为制定安全策略提供依据。
- **确保合规性：**确保软件系统符合相关的安全标准、法规和政策要求，避免因违反规定而面临法律风险和声誉损失。
- **增强安全性：**通过测试和漏洞修复，不断改进软件系统的安全性，提高系统抵御各种安全威胁的能力。

兼容性测试

(Compatibility Testing)

Compatibility Testing



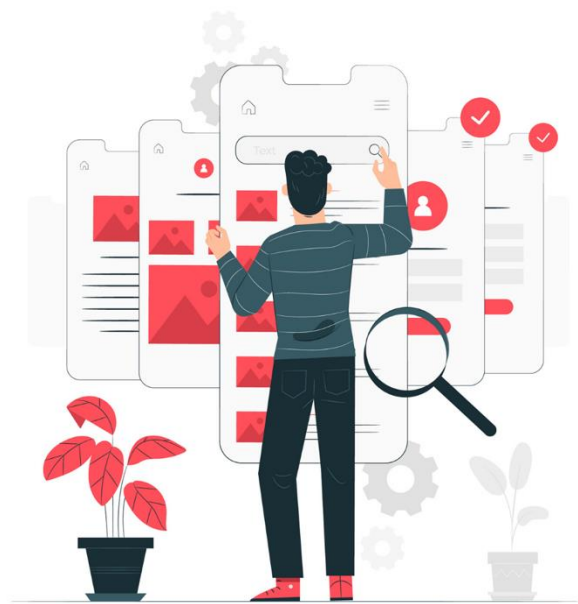
硬件兼容性，操作系统兼容性，浏览器兼容性；数据兼容性；环境多样性，自动化测试，云测试平台

- **确保软件的适用性：**验证软件是否能在各种目标环境中正常工作，包括不同的操作系统、硬件设备、浏览器、数据库等，使软件能满足不同用户的使用需求，提高软件的市场竞争力。
- **提高产品用户体验：**保证软件在不同环境下的稳定性和一致性，避免因兼容性问题导致的软件崩溃、功能异常等情况，为用户提供良好的使用体验，增强用户对软件的满意度和忠诚度。
- **减少技术支持成本：**提前发现并解决兼容性问题，降低软件上线后用户遇到问题的概率，减少技术支持的工作量和成本，避免因兼容性问题引发的用户投诉和业务损失。

适配性测试（Adaptability Testing）是指验证软件在不同设备、操作系统、硬件配置等环境下的适应能力。它不仅关注软件能否运行，还关注软件在不同环境下的表现是否符合设计需求。

易用性测试

(Usability Testing)



USABILITY
TESTING

界面测试，交互测试；功能易用性，用户文档测试；可用性测试，用户体验测试

- **提高用户满意度：**发现并改进软件中可能存在的易用性问题，使用户更轻松高效地使用软件，从而提高用户满意度和忠诚度。
- **提升产品竞争力：**在市场上，易用性良好的软件往往更具竞争力。经过易用性测试优化的软件能够吸引更多用户，有助于软件在同类产品中脱颖而出，提高市场占有率。
- **优化用户体验：**确保软件的界面设计、操作流程、功能布局等符合用户的使用习惯和心理预期，为用户提供良好的体验。
- **降低培训成本：**使软件易于理解 and 操作，减少用户在学习和使用软件过程中所需的培训时间，降低用户培训成本。

回顾与讨论

①

源代码信息依赖程度

白盒测试
灰盒测试
黑盒测试

②

软件开发流程

单元测试
集成测试
系统测试
验收测试

③

软件质量属性

功能测试
性能测试
安全测试
兼容性测试
易用性测试

讨论1：三种分类方法之间的关联？
讨论2：简要介绍有其他分类方法？